OFFLINE · PRIVACY-FIRST · NO CLOUD APIs

Mock-Interview-Hub

Technical Documentation & Setup Guide

“Every great career begins with a confident conversation.”

Practice interviews with an AI coach that helps you think clearly, answer confidently, and grow with every session.

Project	Mock-Interview-Hub
Type	Fully offline interview practice application
Backend	Python · FastAPI · SQLite · SQLAlchemy
Frontend	React · Vite · Tailwind CSS
Speech-to-text	Vosk (on-device, no internet required)
Audio pipeline	FFmpeg → 16 kHz mono PCM WAV → Vosk
Author	Gangarapu Shirisha (@Sirisha-collab)
Document version	1.0

Table of Contents

1	Introduction	3
2	Feature Overview	3
3	System Architecture	4
4	Technology Stack	6
5	Prerequisites	6
6	Project Structure	8
7	Backend Setup (FastAPI)	9
8	Installing the Vosk Speech Model	10
9	Frontend Setup (React + Vite)	11
10	Running the Application	11
11	Data Model	12
12	API Reference	13
13	Using the Application	16
14	Configuration Reference	17
15	Troubleshooting	17
16	Deployment Notes	19
17	Roadmap	19
18	Credits & License	19

1. Introduction

Mock-Interview-Hub is a self-contained interview practice application that runs entirely on your own machine. You maintain a personal bank of interview questions, run timed practice sessions where questions are served one at a time, speak your answers aloud, and get an immediate written transcript of what you said — all without an internet connection and without sending a single second of audio to a third-party service.

The transcription is handled by **Vosk**, an offline speech recognition toolkit. Once the language model is downloaded during setup, every subsequent recording is processed locally. This makes the application well suited to private preparation, environments with restricted network access, and demonstrations where cloud credentials are not available.

Who this document is for

- **Users** who want to install and run the application locally — read sections 5 through 13.
- **Developers** extending the codebase — read sections 3, 6, 11 and 12.
- **Reviewers and evaluators** assessing the project — sections 1 through 4 give the full picture.

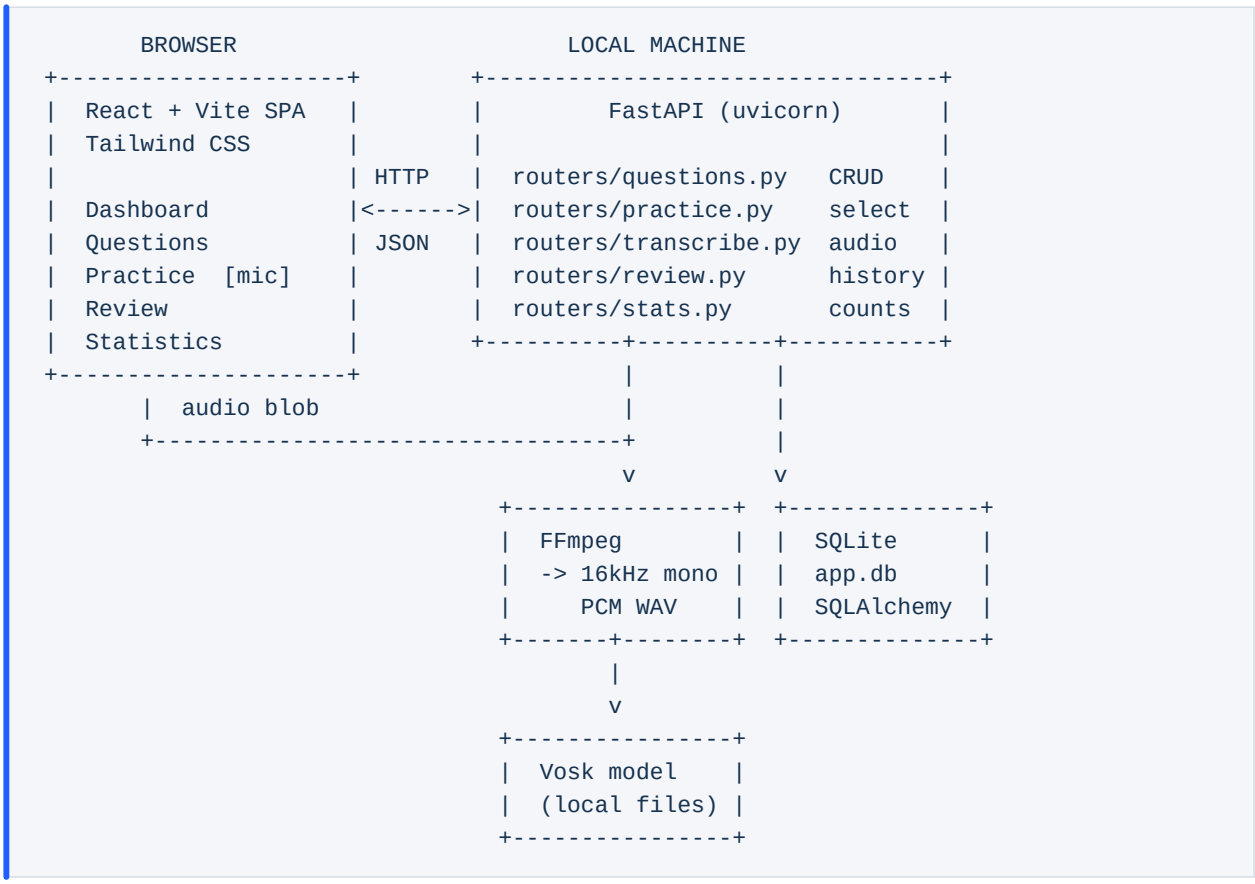
2. Feature Overview

Feature	Description
Question bank	Add, edit, delete and search interview questions from a single management screen.
Categories	Group questions by topic (e.g. Behavioural, Python, System Design) and filter by category using a dropdown.
Difficulty levels	Classify every question as **Easy**, **Medium** or **Hard** so practice can be scaled to your current level.
Bulk import	Insert many questions in a single operation instead of adding them one by one.
Practice sessions	Serves one randomly selected question at a time with **Skip** and **Next** controls, so the same question is not repeated within a session.
Offline recording	Record your spoken answer in the browser and have it transcribed by Vosk immediately after you stop.
Review page	Read back every answer you gave and every question you skipped.
Statistics dashboard	Live counts of total, answered, skipped and unanswered questions.

Privacy: Audio never leaves the machine. The browser sends the recording to your own local FastAPI process, FFmpeg converts it, Vosk transcribes it, and only the resulting text is stored in the local SQLite file.

3. System Architecture

The application follows a conventional two-tier layout: a React single-page application talks to a FastAPI service over HTTP/JSON, and the service owns all persistence and all speech processing. Nothing else is required at runtime.



Request flow for a recorded answer

- The Practice page requests a question that has not yet been served in this session.
- You press record; the browser captures audio using the MediaRecorder API.
- On stop, the audio blob is posted as `multipart/form-data` to the transcription endpoint.
- FastAPI writes the upload to a temporary file and invokes **FFmpeg** to normalise it to 16 kHz, mono, 16-bit PCM WAV — the format the Vosk recogniser expects.
- The WAV stream is fed to Vosk in chunks; partial results are accumulated and the final result is returned as plain text.
- The transcript is saved against the question as an answer, the temporary files are deleted, and the text is returned to the browser for display.

Design decisions

Decision	Rationale
SQLite over a server database	Zero configuration, single file, trivially portable. The workload is single-user, so concurrency limits are irrelevant.
Vosk over a cloud STT API	No API keys, no per-minute cost, no network dependency, and no audio leaves the device.
FFmpeg as a preprocessing step	Browsers record WebM/Opus; Vosk needs PCM WAV. FFmpeg is the most reliable way to bridge the two across all platforms.
Router-per-concern layout	Each feature area lives in its own module, so endpoints stay short and new features can be added without touching existing files.

4. Technology Stack

Layer	Technology	Role
Backend framework	FastAPI	HTTP routing, request validation, automatic OpenAPI docs
ASGI server	Uvicorn	Runs the FastAPI application
Database	SQLite	Single-file persistence for questions and answers
ORM	SQLAlchemy	Models and queries in `models.py` / `database.py`
Validation	Pydantic	Request/response schemas in `schemas.py`
Speech-to-text	Vosk	Offline transcription of recorded answers
Audio conversion	FFmpeg	Normalises browser audio to 16 kHz mono PCM WAV
Frontend library	React	Component-based UI
Build tool	Vite	Dev server with hot reload and production bundling
Styling	Tailwind CSS	Utility-first styling for all components

5. Prerequisites

Install the following once, before anything else.

Tool	Minimum version	Verify with
Python	3.10+	`python3 --version`
Node.js	18+	`node --version`
npm	9+	`npm --version`
FFmpeg	any recent version	`ffmpeg -version`

Installing FFmpeg

FFmpeg is required so the backend can convert your recorded audio into the format Vosk expects. It must be on your PATH.

```
# Windows (PowerShell)
winget install ffmpeg
#   or download a build from https://www.gyan.dev/ffmpeg/builds/
#   and add its \bin folder to your PATH

# macOS
brew install ffmpeg

# Linux (Debian / Ubuntu)
sudo apt install ffmpeg
```

Check: Open a *new* terminal after installing on Windows and run `ffmpeg -version`. If the command is not found, the PATH entry has not been picked up yet.

6. Project Structure

```
interview-practice-app/
|
+-- backend/
|   +-- app/
|   |   +-- main.py            # FastAPI app entrypoint, CORS, router registration
|   |   +-- database.py        # SQLite / SQLAlchemy engine, session factory
|   |   +-- models.py          # ORM models (Question, Answer)
|   |   +-- schemas.py         # Pydantic request / response schemas
|   |   +-- vosk_utils.py      # Vosk model loading + transcription helpers
|   |   +-- routers/
|   |       +-- questions.py   # CRUD, search, categories
|   |       +-- practice.py    # random no-repeat question selection
|   |       +-- transcribe.py  # audio upload -> FFmpeg -> Vosk -> transcript
|   |       +-- review.py      # review past answers
|   |       +-- stats.py       # statistics dashboard
|   +-- requirements.txt
|   +-- vosk-model/            # you create this - see section 8
|
+-- frontend/
    +-- src/
    |   +-- pages/             # Dashboard, Questions, Practice, Review, Statistics
    |   +-- components/        # Sidebar, Card, Badge, Modal, Feedback
    |   +-- api/client.js      # all HTTP calls to the backend
    +-- package.json
    +-- vite.config.js
```

What lives where

Path	Responsibility
`app/main.py`	Creates the FastAPI instance, enables CORS for the Vite dev server, includes every router and creates database tables on startup.
`app/database.py`	Declares the SQLite engine, the session factory and the declarative base. This is the only file that knows the database file path.
`app/models.py`	SQLAlchemy table definitions — see section 11.
`app/schemas.py`	Pydantic models that define the JSON shape of every request and response.
`app/vosk_utils.py`	Loads the Vosk model once at startup and exposes a transcription helper so the model is not reloaded per request.
`app/routers/`	One module per feature area; each exports an APIRouter that main.py includes.
`frontend/src/api/client.js`	Single place where the backend base URL is configured — change it here if you move the backend off port 8000.

7. Backend Setup (FastAPI)

Open a terminal in VS Code with `Ctrl+`` (`Cmd+`` on macOS) and run the following.

7.1 Create and activate a virtual environment

```
cd backend

python3 -m venv venv

# macOS / Linux
source venv/bin/activate

# Windows (PowerShell)
venv\Scripts\Activate.ps1
```

Your prompt should now be prefixed with `(venv)`. Every command in the rest of this section assumes the environment is active.

> **Windows PowerShell:** If activation fails with an execution-policy error, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` in the same window and try again. This affects only that terminal session.

7.2 Install dependencies

```
pip install -r requirements.txt
```

This installs FastAPI, Uvicorn, SQLAlchemy, Pydantic and the Vosk Python bindings. The first install may take a couple of minutes while the Vosk wheel is fetched.

7.3 Verify the install

```
python -c "import fastapi, sqlalchemy, vosk; print('backend deps OK')"
ffmpeg -version | head -n 1
```

8. Installing the Vosk Speech Model

Vosk needs a language model on disk. The model is **not** bundled with the repository because of its size, so you download it once. After this step the application never needs the internet again.

8.1 Choose a model

Model	Size	When to use it
vosk-model-small-en-us	~40 MB	Recommended default. Fast, low memory, accurate enough for practice transcripts.
vosk-model-en-us (large)	~1.8 GB	Noticeably better accuracy on long or technical answers; needs more RAM and a slower first load.

8.2 Install it

- Download a model from `https://alphacephei.com/vosk/models`.
- Unzip the archive.
- Place the *contents* of the unzipped folder into `backend/vosk-model/`.

```
cd backend
mkdir -p vosk-model
# unzip your download, then move its contents in, so that you end up with:

backend/vosk-model/
+-- am/
+-- conf/
+-- graph/
+-- ivector/
+-- README
```

Common mistake: Do not leave a nested folder such as `vosk-model/vosk-model-small-en-us-0.15/am/`. The `am/`, `conf/` and `graph/` directories must sit *directly* inside `vosk-model/`, otherwise the model will fail to load.

8.3 Confirm the model loads

```
python -c "from vosk import Model; Model('vosk-model'); print('vosk model OK')"
```

9. Frontend Setup (React + Vite)

```
cd frontend
npm install
```

This installs React, Vite, Tailwind CSS and the supporting tooling declared in `package.json`. All HTTP calls are centralised in `src/api/client.js`; if you run the backend on a port other than 8000, update the base URL there.

10. Running the Application

The application needs **two terminals** running at the same time.

Terminal 1 — backend

```
cd backend
source venv/bin/activate          # Windows: venv\Scripts\Activate.ps1
uvicorn app.main:app --reload --port 8000
```

Terminal 2 — frontend

```
cd frontend
npm run dev
```

Service	URL	Notes
Frontend (Vite)	`http://localhost:5173`	Open this in your browser
Backend (FastAPI)	`http://localhost:8000`	JSON API
Interactive API docs	`http://localhost:8000/docs`	Swagger UI, generated automatically
Alternative API docs	`http://localhost:8000/redoc`	ReDoc view of the same schema

Microphone access: Browsers only grant microphone permission on a secure origin. `localhost` counts as secure, so the dev server works as-is — but if you expose the frontend on a LAN IP address, you will need HTTPS for recording to work.

11. Data Model

Two tables carry the entire application state, both stored in a single SQLite file created automatically on first run.

questions

Column	Type	Description
`id`	Integer, PK	Auto-incrementing identifier
`text`	String	The interview question itself
`category`	String	Topic grouping, e.g. Behavioural, Python, SQL, System Design
`difficulty`	String	One of Easy, Medium or Hard
`created_at`	DateTime	Timestamp of insertion

answers

Column	Type	Description
`id`	Integer, PK	Auto-incrementing identifier
`question_id`	Integer, FK	References `questions.id`
`transcript`	Text	The Vosk transcription of the spoken answer
`skipped`	Boolean	True when the question was skipped rather than answered
`created_at`	DateTime	When the attempt was recorded

How the statistics are derived

Metric	Definition
Total	Row count of `questions`
Answered	Distinct questions with at least one non-skipped answer
Skipped	Distinct questions whose only attempts were skips
Unanswered	Questions with no attempts at all

12. API Reference

All endpoints are served from `http://localhost:8000` and return JSON unless noted. The live, always-accurate contract is available at `/docs` while the backend is running — FastAPI generates it from the Pydantic schemas, so it can never drift from the code.

12.1 Questions

Method	Path	Purpose
GET	`/questions`	List questions; supports search and category filtering
POST	`/questions`	Create a single question
POST	`/questions/bulk`	Insert many questions in one request
GET	`/questions/{id}`	Fetch one question
PUT	`/questions/{id}`	Update text, category or difficulty
DELETE	`/questions/{id}`	Delete a question
GET	`/questions/categories`	Distinct category list used to populate the dropdown

Create a question

```
POST /questions
Content-Type: application/json

{
  "text": "Tell me about a time you disagreed with a teammate.",
  "category": "Behavioural",
  "difficulty": "Medium"
}
```

Bulk import

```
POST /questions/bulk
Content-Type: application/json

{
  "questions": [
    { "text": "What is a Python decorator?", "category": "Python",
      "difficulty": "Medium" },
    { "text": "Explain database indexing.", "category": "SQL",
      "difficulty": "Hard" },
    { "text": "Describe your ideal team.", "category": "Behavioural",
      "difficulty": "Easy" }
  ]
}
```

12.2 Practice

Method	Path	Purpose
GET	`/practice/next`	Return a random question not yet served in this session
POST	`/practice/skip`	Record the current question as skipped and move on

Selection is random but non-repeating: identifiers already served are excluded until the pool is exhausted, at which point the session reports that every question has been seen.

12.3 Transcription

Method	Path	Purpose
POST	`/transcribe`	Upload recorded audio and receive the transcript

```
POST /transcribe
Content-Type: multipart/form-data

file=@answer.webm
question_id=12

-> 200 OK
{
  "question_id": 12,
  "transcript": "i once disagreed with a teammate about the deployment schedule ...",
  "duration_seconds": 41.2
}
```

> **Note:** Transcription is synchronous — the request returns once Vosk has finished. A long answer on the large model can take several seconds, so keep the frontend's request timeout generous.

12.4 Review and statistics

Method	Path	Purpose
GET	`/review`	All past attempts, answered and skipped, most recent first
GET	`/stats`	Total, answered, skipped and unanswered counts

```
GET /stats
-> 200 OK
{
  "total": 48,
  "answered": 31,
  "skipped": 6,
  "unanswered": 11
}
```

13. Using the Application

Screen	What it does
Dashboard	Landing screen with a summary of your question bank and quick links into practice.
Questions	Full management of the question bank: add, edit, delete, search, filter by category, set difficulty and run a bulk import.
Practice	Serves one question at a time. Record your answer, or use **Skip** to pass and **Next** to advance. The transcript appears as soon as processing finishes.
Review	A chronological list of everything you have answered or skipped, with the transcript shown alongside each question.
Statistics	Counts of total, answered, skipped and unanswered questions so you can see coverage at a glance.

A suggested first session

- Open **Questions** and bulk-import 20–30 questions covering the roles you are targeting.
- Tag each one with a category and a difficulty so you can practise selectively later.
- Go to **Practice**, allow microphone access when prompted, and answer out loud as you would in a real interview — do not stop to correct yourself.
- Use **Skip** honestly. A skipped question is a data point, and it will show up in your statistics as a gap worth closing.
- Afterwards, open **Review** and read your transcripts. Filler words, circular sentences and unfinished thoughts are far easier to spot in text than they are to hear.
- Re-run the same questions a few days later and compare the transcripts.

Getting better transcripts

Factor	Guidance
Microphone	A headset or external microphone gives a noticeably cleaner signal than a laptop's built-in array.
Room	Record somewhere quiet; background speech confuses the recogniser more than steady noise does.
Pace	Speak at a normal conversational pace. Rushing hurts accuracy more than pausing does.
Model size	If technical vocabulary is being mangled, switch to the large model — the accuracy difference on domain terms is significant.

14. Configuration Reference

Setting	Default	Where to change it
Backend port	8000	`--port` flag on the uvicorn command
Frontend port	5173	`server.port` in `vite.config.js`
API base URL	http://localhost:8000	`frontend/src/api/client.js`
Database file	SQLite file in `backend/`	`app/database.py`
Vosk model path	`backend/vosk-model/`	`app/vosk_utils.py`
Allowed origins	Vite dev server	CORS middleware in `app/main.py`

Resetting the application

Because everything is stored in a single SQLite file, a full reset is just a file deletion. Back it up first if you want to keep your transcripts — the tables are recreated automatically the next time the backend starts.

```
# from backend/ — back up first, then remove
cp app.db app.db.backup
rm app.db
```

15. Troubleshooting

Symptom	Likely cause and fix
ffmpeg not found when transcribing	FFmpeg is not installed or not on the PATH. Install it (section 5), then open a *new* terminal and restart uvicorn so it inherits the updated environment.
Backend fails at startup with a Vosk model error	The model directory is nested or empty. Confirm that `am/`, `conf/` and `graph/` sit directly inside `backend/vosk-model/` (section 8).
Transcript comes back empty	Usually no audio was captured. Check the browser granted microphone permission, that the correct input device is selected in your OS settings, and that the recording is longer than a second or two.
Frontend loads but every request fails	The backend is not running, is on a different port, or CORS is not permitting the frontend origin. Check `http://localhost:8000/docs` loads, then verify the base URL in `src/api/client.js`.
Address already in use on port 8000	Another process holds the port. Either stop it, or start uvicorn with `--port 8001` and update the frontend base URL to match.

Symptom	Likely cause and fix
PowerShell refuses to activate the venv	Execution policy. Run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` in that terminal and activate again.
Transcription is very slow	Expected on the large model, especially on the first request while the model loads into memory. Switch to the small model if latency matters more than accuracy.
Accuracy is poor on technical terms	A known limitation of the small model. Move to the large model, and improve microphone quality — input quality dominates recogniser choice.

16. Deployment Notes

The application is designed to run locally, and that is where it works best — the privacy argument for offline speech recognition disappears the moment the audio crosses a network. If you do want to run it on another machine on your own network, keep the following in mind.

- Build the frontend with `npm run build` and serve the generated `dist/` directory as static files.
- Run uvicorn without `--reload` in any non-development setting.
- Browsers require a secure context for microphone access, so anything beyond `localhost` needs HTTPS.
- SQLite is single-writer. It is entirely adequate for one user and not intended for a shared, multi-user deployment.
- The Vosk model must be present on whichever machine runs the backend; it is the largest part of the deployment footprint.

17. Roadmap

Natural next steps for the project, in rough order of effort:

Idea	Sketch
Answer timing	Record and display how long each answer took, and flag responses that run past a target duration.
Filler-word analysis	Count occurrences of 'um', 'like', 'you know' in the transcript and trend them across sessions.
Session history	Group attempts into explicit sessions so progress can be compared run over run rather than answer by answer.
Export	Download transcripts and statistics as CSV or PDF for offline review.
Local model feedback	Run a small local language model over each transcript to suggest structural improvements, keeping the fully-offline guarantee intact.
Spaced repetition	Resurface questions you skipped or answered weakly more often than ones you handled well.

18. Credits & License

Author	Gangarapu Shirisha — `@Sirisha-collab`
Languages	JavaScript 68.9% · Python 28.9% · CSS 1.4% · HTML 0.8%

Speech engine	Vosk, by Alpha Cephei
Audio tooling	FFmpeg
Document	Mock-Interview-Hub Technical Documentation, version 1.0

Every great career begins with a confident conversation.